

# Day 4 Practice Assignments: Dictionaries & Exception Handling

---

## Objective

Model key-value storage relations, implement robust custom exceptions, and design atomic transaction processing using try-except-finally blocks.

---

## Easy Assignments

### Assignment 1: Inventory Tracker for CDAC Bookstore

#### Scenario

The CDAC Bookstore needs a backend helper module to manage books and their quantities. The inventory is stored in a Python dictionary where keys are book titles (strings) and values are quantities in stock (non-negative integers).

#### Problem Description

Write a function `manage_bookstore_inventory(inventory, action, book_title, quantity=0)` that handles inventory operations safely.

1. The `action` parameter can be one of three options: `"add"`, `"sell"`, or `"lookup"`.
2. **Add Action ("`add`"):**
  - Add the specified `quantity` to the existing stock of `book_title`.
  - If the book is not in the inventory dictionary, add it as a new key with `quantity` as the value.
3. **Sell Action ("`sell`"):**
  - Decrease the stock of `book_title` by the specified `quantity`.
  - If the book is not found in the inventory, print a message: `Error: Book '<book_title>' not found in inventory.` and make no changes. (Do not let the program crash with a `KeyError`).
  - If the requested quantity to sell exceeds the stock available, print: `Error: Insufficient stock for '<book_title>'. Available: <current_stock>.` and make no changes.
  - If the stock reaches exactly `0` after a successful sale, remove the book key from the inventory entirely.
4. **Lookup Action ("`lookup`"):**
  - Look up the stock quantity of `book_title` and return it.
  - Use safe dictionary retrieval; if the book does not exist, return `0` without throwing a `KeyError`.

The function must return the updated/current inventory dictionary.

#### Example Walkthrough

```
# Initial Inventory
inventory = {"Python Basics": 10, "Learning AI": 5}

# 1. Add Stock
inventory = manage_bookstore_inventory(inventory, "add", "Python Basics",
5)
# Result: {"Python Basics": 15, "Learning AI": 5}

# 2. Sell Stock Safely (Missing Book)
inventory = manage_bookstore_inventory(inventory, "sell", "Data Science
101", 1)
# Console output: Error: Book 'Data Science 101' not found in inventory.

# 3. Sell Stock (Insufficient)
inventory = manage_bookstore_inventory(inventory, "sell", "Learning AI",
10)
# Console output: Error: Insufficient stock for 'Learning AI'. Available:
5.

# 4. Sell Stock (Exactly Zero Stock)
inventory = manage_bookstore_inventory(inventory, "sell", "Learning AI", 5)
# Result: {"Python Basics": 15}
```

---

## Assignment 2: Robust Phonebook Contact Registry

### Scenario

You are writing a Command-Line Interface (CLI) contact registry that maps user names to their phone numbers. The program needs to validate user inputs robustly to prevent corrupted formatting or empty values from breaking the registry database.

### Problem Description

1. Define a custom exception class named `InvalidPhoneNumberError` that inherits from `Exception`.
2. Write a function `register_contact(phonebook, name, phone_input)`:
  - `phonebook` is a dictionary mapping contact names (strings) to their phone numbers (strings).
  - Validate the `name` parameter: it must be a non-empty string consisting only of alphabetic characters and spaces. If invalid, raise a standard `ValueError` with the message: `"Contact name must be a non-empty alphabetic string."`
  - Validate the `phone_input` parameter: it must consist only of digits. Check this by attempting to convert it to an integer using `int()`.
    - If the conversion fails (raises a `ValueError`), catch that exception and raise your custom `InvalidPhoneNumberError` with the message: `"Phone number must contain digits only."`

- If validations pass, store `phone_input` as a string in the `phonebook` under the key `name` (preserving any leading zeros).
- Return the updated `phonebook` dictionary.

## Example Walkthrough

```
contacts = {}

# 1. Valid Input
contacts = register_contact(contacts, "Alice", "0987654321")
# Result: {"Alice": "0987654321"}

# 2. Invalid Phone Number (Raises InvalidPhoneNumberError)
try:
    contacts = register_contact(contacts, "Bob", "123-456-789")
except InvalidPhoneNumberError as e:
    print(e) # Output: Phone number must contain digits only.

# 3. Invalid Name (Raises ValueError)
try:
    contacts = register_contact(contacts, "Bob123", "9876543210")
except ValueError as e:
    print(e) # Output: Contact name must be a non-empty alphabetic string.
```

---

## Medium Assignments

### Assignment 3: Course Feedback Compiler & Sanitizer

#### Scenario

Student feedback records contain ratings from 1 to 5 stars. Due to raw data entry issues, the feedback database has some course entries with list values that are empty, or lists containing invalid elements (such as string annotations like "Excellent" or None values).

#### Problem Description

Write a function `compile_feedback(ratings_dict)` that processes course feedback:

- The parameter `ratings_dict` is a dictionary where keys are course names (strings) and values are lists of ratings (which should be numeric but may contain invalid types).
- The function must return a dictionary mapping each course name to its average rating, rounded to **2 decimal places**.
- Implement the following error handling criteria:
  1. For each rating inside a course's list, attempt to convert it to a `float`. If a rating cannot be converted (throws a `ValueError` or `TypeError`), catch the exception, print a warning: `"Warning: Invalid rating value '<val>' in course '<course>' skipped."`, and continue processing the rest of the list.

2. If a course has no valid ratings (the list is empty or contains no convertible numbers), computing the average will trigger a division-by-zero error. Catch `ZeroDivisionError`, print a warning: `"Warning: No valid ratings found for course '<course>'. Rating set to 0.0."`, and assign the course an average rating of `0.0`.

### Sample Input

```
feedback_data = {  
    "Python Programming": [5, 4, "4", "Great", 5],  
    "Machine Learning": [],  
    "Deep Learning": ["Good", "Average", None]  
}
```

### Expected Output

#### Console Warnings Printed:

```
Warning: Invalid rating value 'Great' in course 'Python Programming'  
skipped.  
Warning: No valid ratings found for course 'Machine Learning'. Rating set  
to 0.0.  
Warning: Invalid rating value 'Good' in course 'Deep Learning' skipped.  
Warning: Invalid rating value 'Average' in course 'Deep Learning' skipped.  
Warning: Invalid rating value 'None' in course 'Deep Learning' skipped.  
Warning: No valid ratings found for course 'Deep Learning'. Rating set to  
0.0.
```

#### Returned Dictionary:

```
{  
    "Python Programming": 4.5,  
    "Machine Learning": 0.0,  
    "Deep Learning": 0.0  
}
```

---

## Assignment 4: Atomic E-Commerce Order Processor

### Scenario

You are building an ordering subsystem for an online store. Orders containing multiple products must be processed **atomically**: either the entire order completes successfully, or the entire transaction fails. If one item in the order is out of stock or is unrecognized, no stock should be deducted for any other item (rollback).

## Problem Description

1. Define two custom exceptions:
  - `ProductNotFoundError` (raised when a product ID is not present in the catalog).
  - `OutOfStockError` (raised when the customer's ordered quantity exceeds the available stock).

2. Write a function `process_order(catalog, order)`:

- `catalog` is a dictionary containing product database records. Format:

```
catalog = {  
    "P01": {"price": 100.0, "stock": 5},  
    "P02": {"price": 50.0, "stock": 2}  
}
```

- `order` is a dictionary containing product IDs (keys) and quantities ordered (values). Format: `{"P01": 2, "P02": 1}`.
- **Validation Phase:** Before modifying any inventory levels:
  - Check if all ordered keys exist in the catalog. If a product ID does not exist, raise `ProductNotFoundError` with message: `"Product '<product_id>' not found in store catalog."`
  - Check if the catalog contains sufficient stock for each item ordered. If the ordered quantity exceeds available stock, raise `OutOfStockError` with message: `"Product '<product_id>' is out of stock. Requested: <requested_qty>, Available: <available_stock>."`
- **Execution Phase:** If (and only if) all products pass validation:
  - Deduct the ordered quantities from the stock numbers in the catalog dictionary.
  - Calculate and return the total cost of the order (float).
  - If an exception was raised during validation, the catalog must remain completely unchanged.

## Example Walkthrough

```
catalog = {  
    "P01": {"price": 10.0, "stock": 5},  
    "P02": {"price": 20.0, "stock": 10}  
}  
  
# 1. Successful Order  
total = process_order(catalog, {"P01": 2, "P02": 1})  
# Returns: 40.0  
# Catalog stock changes to: P01 stock = 3, P02 stock = 9  
  
# 2. Failed Order (Triggers Rollback)  
# Current Catalog: {"P01": {"price": 10.0, "stock": 3}, "P02": {"price":  
20.0, "stock": 9}}  
try:  
    total = process_order(catalog, {"P01": 2, "P02": 15})
```

```
except OutOfStockError as e:
    print(e) # Output: Product 'P02' is out of stock. Requested: 15,
    Available: 9.

# Verify Catalog Stock: P01 must remain at 3 (NOT decreased to 1).
print(catalog["P01"]["stock"]) # Output: 3
```

## Difficult Assignments

### Assignment 5: Deep JSON/Configuration Key Traverser

#### Scenario

Configuration files loaded from JSON databases consist of nested dictionary hierarchies. Checking key existence at every level using nested conditions (`if key in dictionary`) leads to complex and verbose code. You need to write a clean traverser utility that navigates nested dictionaries using exceptions.

#### Problem Description

Write a function `traverse_nested_config(config_dict, path_str, default=None)`:

- `config_dict` is a nested dictionary configuration tree.
- `path_str` is a string specifying the configuration path using dot notation (e.g., `"server.database.port"`).
- The function should split the `path_str` on `.` characters and traverse down `config_dict`.
- **Implementation Constraint:** You **must** attempt to traverse keys directly. Do not use key-existence checks (like `if key in dict`) or class-checks (like `if isinstance(sub_dict, dict)`). Instead, handle the lookup path directly inside a `try` block and catch the following exceptions to return the `default` value:
  - Catch `KeyError` if any key in the path does not exist.
  - Catch `TypeError` or `AttributeError` if you try to index a primitive, non-dictionary value (e.g., trying to access a key like `"port"` on a configuration value that resolved to a string or number).
- If `path_str` is empty or `config_dict` is not a valid dictionary, return the `default` value.

#### Test Data & Test Cases

```
config = {
    "server": {
        "host": "127.0.0.1",
        "port": 8080,
        "ssl": {
            "enabled": True,
            "cert_path": "/etc/ssl/certs"
        }
    },
    }
```

```

    "database": "postgresql://localhost:5432"
}

# Test Case 1: Valid Path
print(traverse_nested_config(config, "server.ssl.cert_path"))
# Output: /etc/ssl/certs

# Test Case 2: Missing Key (Triggers KeyError)
print(traverse_nested_config(config, "server.database.username", "guest"))
# Output: guest

# Test Case 3: Indexing Non-Dictionary value (Triggers TypeError)
# Here config["database"] is a string, which cannot be indexed with "host"
print(traverse_nested_config(config, "database.host", "localhost"))
# Output: localhost

```

## Assignment 6: Atomic Transaction processing with Log Rollback

### Scenario

A bank updates user balances in a database dictionary based on transaction files. To ensure accounting consistency, if *any* single transaction in a batch contains an error (such as a negative transfer amount, an unrecognized account number, or an overdraft), the *entire batch* must fail, all accounts must be restored to their initial states, and a rollback action must be logged to a text file.

### Problem Description

1. Define three custom exception classes inheriting from `Exception`:
  - `AccountNotFoundError` (raised when an account ID is missing from the registry).
  - `OverdraftError` (raised when a withdrawal amount exceeds the account balance).
  - `InvalidTransactionError` (raised when the transaction type is unrecognized or if transaction amounts are non-positive).
2. Write a function `process_transaction_batch(accounts, batch_list, log_path)`:
  - `accounts` is a dictionary where keys are account numbers (strings) and values are balances (floats), e.g., `{"ACC01": 500.0, "ACC02": 200.0}`.
  - `batch_list` is a list of dictionaries representing transactions, e.g.:

```

[
    {"acc": "ACC01", "type": "deposit", "amt": 150.0},
    {"acc": "ACC02", "type": "withdraw", "amt": 50.0}
]

```

- `log_path` is a string referencing the path of the transaction log file.
- **Atomicity Requirements:**
  - Create a deep copy of the `accounts` dictionary before starting any transaction modifications to act as a restore point (backup).

- Iterate through `batch_list` and apply the changes to `accounts`:
  - If the transaction `"acc"` does not exist in `accounts`, raise `AccountNotFoundError` with message: `"Account '<acc>' not found."`
  - If transaction `"type"` is not `"deposit"` or `"withdraw"`, raise `InvalidTransactionError` with message: `"Invalid transaction type '<type>'."`
  - If transaction `"amt"` is less than or equal to `0`, raise `InvalidTransactionError` with message: `"Transaction amount must be positive."`
  - If transaction `"type"` is `"withdraw"` and the account balance is less than `"amt"`, raise `OverdraftError` with message: `"Insufficient funds. Account <acc> has balance <bal>, requested <amt>."`
- **Exception Handling & Rollback:**
  - If any exception is raised during the processing of the list, catch the exception:
    - Restore the `accounts` dictionary to the exact state saved in your backup.
    - Open the file at `log_path` (create it if it doesn't exist, append to it if it does) and write the following entry: `[ROLLBACK] Batch aborted: <Exception Class Name> – <Exception Message>\n`
    - Re-raise the caught exception so that the calling program knows the transaction batch failed.
  - If all transactions in the batch are executed successfully:
    - Open the file at `log_path` and write: `[SUCCESS] Batch completed. <number_of_transactions> transaction(s) processed.\n`
    - Return the updated `accounts` dictionary.
- **Constraint:** Ensure all file operations are safely cleaned up. Use context managers (`with open(...)`) or `try...finally` to write to the log file.

## Example Walkthrough

```
accounts = {"ACC01": 100.0, "ACC02": 50.0}
log_file = "transactions.log"

# Batch 1: Valid transactions
batch_1 = [
    {"acc": "ACC01", "type": "withdraw", "amt": 30.0},
    {"acc": "ACC02", "type": "deposit", "amt": 20.0}
]
accounts = process_transaction_batch(accounts, batch_1, log_file)
# Result: accounts changes to {"ACC01": 70.0, "ACC02": 70.0}
# transactions.log writes: "[SUCCESS] Batch completed. 2 transaction(s) processed."

# Batch 2: Invalid transaction (triggers rollback)
batch_2 = [
    {"acc": "ACC01", "type": "deposit", "amt": 50.0},
    {"acc": "ACC02", "type": "withdraw", "amt": 200.0} # Overdraft!
]
try:
```



```
accounts = process_transaction_batch(accounts, batch_2, log_file)
except OverdraftError as e:
    print(f"Caught: {e}")

# Verify Rollback: ACC01 must remain 70.0, NOT updated to 120.0.
print(accounts) # Output: {"ACC01": 70.0, "ACC02": 70.0}
# transactions.log writes: "[ROLLBACK] Batch aborted: OverdraftError -
Insufficient funds. Account ACC02 has balance 70.0, requested 200.0."
```